

PART VI — THE SELF-LEARNING LAYER

Chapter 34

Chunk-Level Deduplication and Merging During Retrieval

*“The same fact retrieved three times by three different queries is not three facts.
Treating it as one is where retrieval-time merging earns its keep.”*

Chapter 34 — Chunk-Level Deduplication and Merging During Retrieval

This chapter walks through the retrieval-time deduplication and merging logic living inside ``_accumulate_track()`` in ``agent_query.py`` — a genuinely careful piece of engineering hiding behind a deceptively short block of code, with a specific similarity threshold, a specific clustering strategy, an LLM-based merge step with real failure handling, and a specific ordering discipline that exists to prevent a real and easy-to-introduce class of bug.

34.1 Why retrievals across iterations produce near-duplicate chunks

``_PROCESS_INSTRUCTIONS`` deliberately asks for "SHORT, semantically different queries" across 2-3 retrieval calls per run (Chapter 26.3) — different angles on the same underlying topic, by design. Different angles on the same topic very often retrieve overlapping source material: a query about "ASD diagnosis criteria" and a separate query about "ASD screening tools" can both legitimately surface the same paragraph if that paragraph happens to discuss both. Each individual retrieval call is behaving correctly; the redundancy is an emergent property of running several correct, independent searches against one corpus and accumulating everything they return.

Left unmerged, this redundancy compounds every cost Part V already quantified — Chapter 23.4's token math, Chapter 24's attention dilution — for zero added information. A chunk retrieved twice does not give the model twice as much evidence; it gives the model the identical evidence twice, at twice the token cost, with no compensating benefit.

It is worth distinguishing this problem explicitly from the one Chapter 22's NAC compression stage solves, since both operate on chunk redundancy but at different points in the pipeline and for different reasons. NAC merges **consecutive** chunks from the **same** source document, restoring document flow that chunking (Chapter 7) broke apart — a structural, position-based redundancy. This chapter's retrieval-time merge catches a **content-based** redundancy that can span entirely different source documents and different retrieval calls, with no positional relationship between the duplicated chunks at all. The two mechanisms are complementary, not overlapping: this chapter's merge runs first, during accumulation; NAC runs later, during the compression pipeline proper, on whatever survived accumulation unmerged.

34.2 The `MERGE_SIMILARITY_THRESHOLD` knob

``MERGE_SIMILARITY_THRESHOLD = 0.90`` is the single number deciding whether two accumulated chunks are similar enough to merge into one. This retrieval-time threshold is deliberately distinct from — and stricter in spirit than — the embedding-first prefiltering threshold ADR-007 identifies for a different stage of the pipeline (DC's redundancy prefiltering, $\cosine \geq 0.92$ candidate-narrowing before an LLM judge call): the two thresholds serve different purposes even though both operate on cosine similarity between chunk embeddings.

A threshold set too low (0.85, for instance) risks merging chunks that are topically related but substantively distinct — collapsing genuinely different facts into one merged chunk, exactly the failure mode Chapter 22B's redundancy-judge prompt guards against explicitly ("related topic, but different facts" is a REJECTED verdict, not a CONFIRMED one, in that judge's own worked examples). A threshold set too high (0.95+) risks merging almost nothing, leaving genuine near-duplicates — the same fact restated with minor wording differences across two retrieval calls — sitting in the accumulated pool unmerged, forfeiting the token savings this mechanism exists to capture. 0.90 sits deliberately between those two failure directions: tight enough to require near-paraphrase-level similarity, loose enough to actually catch the common case of the identical passage surfacing from two different query angles.

34.3 Cosine similarity at retrieval time

The comparison in `_accumulate_track()` runs on `accumulated_chunks` — every chunk retrieved so far across the whole run, not merely the chunks from the current call — using `retriever.embedding_manager.cosine_similarity()` on each chunk's stored `embedding` field. This embedding is computed once, at the moment a chunk first enters the accumulated pool, and reused for every subsequent comparison against it rather than being recomputed on each new incoming chunk.

```
accumulated_chunks.append({
    "content": content,
    "source": source,
    **({"embedding":
retriever.embedding_manager.generate_embedding(content)}
    if ENABLE_RETRIEVAL_DEDUP_MERGE else {}),
    **({"chunk_seq": chunk_seq} if isinstance(chunk_seq, int) else {}),
})
```

Gating embedding computation behind `ENABLE_RETRIEVAL_DEDUP_MERGE` (Chapter 36B.2) matters for cost, not just code cleanliness: an embedding call for every accumulated chunk is real, measurable latency, and a deployment that has decided deduplication is not worth its cost should not pay for the embeddings dedup would have needed either.

34.4 Greedy, star-shaped merging — Not single-link clustering

The clustering algorithm in `_accumulate_track()` is a specific, deliberate choice, not the only reasonable one. For each unclaimed chunk index `i`, it forms a group by comparing `i` against every later unclaimed index `j`, claiming any `j` whose similarity to `i` clears the threshold — a greedy, star-shaped cluster centered on `i`. This is meaningfully different from single-link (chain) clustering, where `A` similar to `B` and `B` similar to `C` would transitively group `A` with `C` even if `A` and `C` are not directly similar to each other at all.

```
claimed: set[int] = set()
merge_groups: list[list[int]] = []
for i in range(len(accumulated_chunks)):
    if i in claimed:
        continue
    group = [i]
    for j in range(i + 1, len(accumulated_chunks)):
        if j in claimed:
            continue
        sim = cosine_similarity(accumulated_chunks[i]["embedding"],
accumulated_chunks[j]["embedding"])
        if sim >= MERGE_SIMILARITY_THRESHOLD:
            group.append(j)
            claimed.add(j)
    if len(group) > 1:
        claimed.add(i)
        merge_groups.append(group)
```

Figure 34.1 draws the contrast that matters most between the two clustering strategies directly: a star-shaped group's every member has a verified direct relationship to its leader, while the single-link alternative shown alongside it would permit a chain of only pairwise-adjacent similarities to stand in for group-wide relatedness that was never actually confirmed.

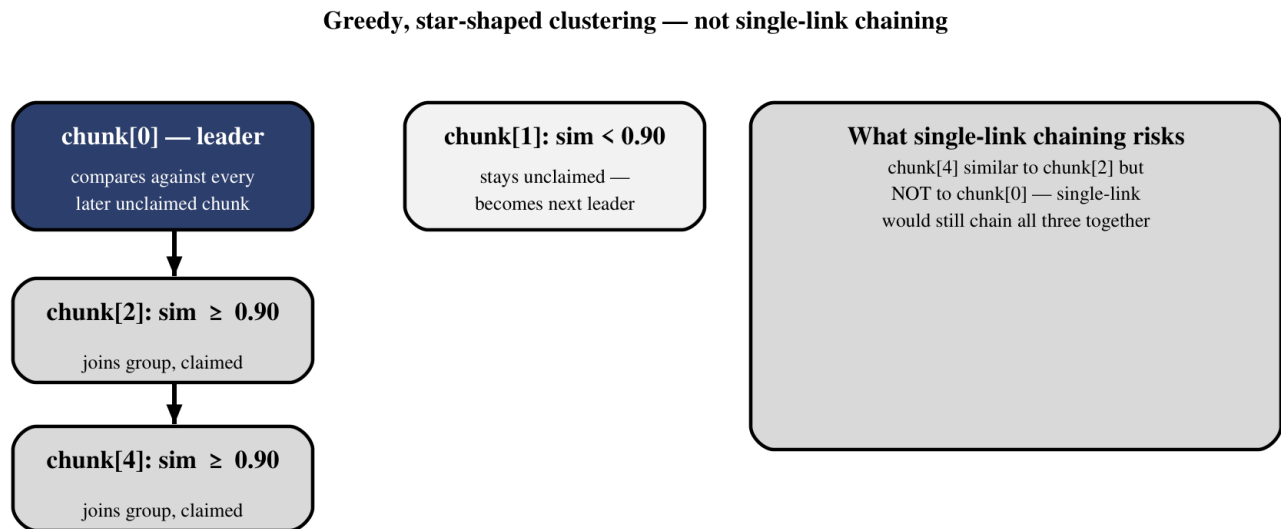


Figure 34.1 — Every member of a group is directly similar to the group's leader; single-link chaining would allow indirectly-related chunks to merge, which this design deliberately avoids.

The star-shaped design is a direct, load-bearing consequence of Chapter 22.7's faithfulness concern: every chunk merged into one group is guaranteed to be directly, individually similar to the group's leader, which bounds how far the LLM merge step (Section 34.5) can be asked to reconcile content that might not actually belong together. Single-link clustering would offer no such guarantee — a transitively-chained group could contain a leader and a tail member with no direct similarity to each other at all, asking the merge LLM to faithfully combine content that was never actually shown to be alike.

34.5 The `_merge_similar_chunks` LLM merge step

Each confirmed group is handed to `_merge_similar_chunks()` — the same function this project's own bug ledger (BUG-023) flags as shared infrastructure between retrieval-time dedup and NAC compression, used by two different callers that must both be considered whenever its logic changes. It builds `_CHUNK_MERGE_PROMPT` from the group's chunks, normalizes Windows backslash paths in source strings before JSON-encoding them (a real, specific fix — `_normalize_source()` — for a real bug where literal backslashes broke JSON parsing), and calls the merge LLM with retry.

The failure path is explicit and conservative: `except (json.JSONDecodeError, KeyError, AttributeError, TypeError)` catches any parse failure and falls back to `similar_chunks[0]` — keeping the first chunk in the group, unmerged, discarding the rest of the group's content entirely rather than risking a malformed or partially-fabricated merge result reaching the accumulated pool. This mirrors Chapter 22B's over-compression guard in spirit: when a repair step cannot produce something trustworthy, the safe fallback is to lose some information cleanly, not to keep something unverifiable.

34.6 The mutation-during-iteration bug — and how the code avoids it

A tempting but unsafe implementation would delete merged-away chunks from `accumulated_chunks` as soon as each group is resolved — `del accumulated_chunks[j]` inside the same loop that is still scanning the list by index. This is the classic mutate-while-iterating bug: deleting an element shifts every subsequent index down by one, so a loop still using its original index values silently skips elements or compares the wrong pair, and the corruption is often invisible unless specifically tested for.

This project's code avoids the entire failure class structurally rather than through careful index bookkeeping: `indices_to_drop: set[int]` accumulates every index the merge groups decided to discard, and the actual list is only ever rebuilt once, after every group has been resolved — `[c for i, c in enumerate(accumulated_chunks) if i not in indices_to_drop]`. No deletion ever happens mid-scan; the scan and the mutation are two fully separate passes.

The clustering loop itself follows the identical discipline for a second, related reason: `claimed` is a `set[int]`, checked but never used to physically remove anything from `accumulated_chunks` during the scan. Both the clustering pass (deciding which chunks belong to which group) and the merge-application pass (actually discarding merged-away chunks) read the list's positions as fixed and stable throughout their own execution, and only ever act on that list's actual contents in a step cleanly separated from the reading.

Figure 34.2 puts the unsafe pattern and this project's actual pattern side by side specifically so the difference is visible as a structural one — not a matter of being more careful with the same approach, but of choosing an approach where the dangerous mid-scan mutation is never possible in the first place.

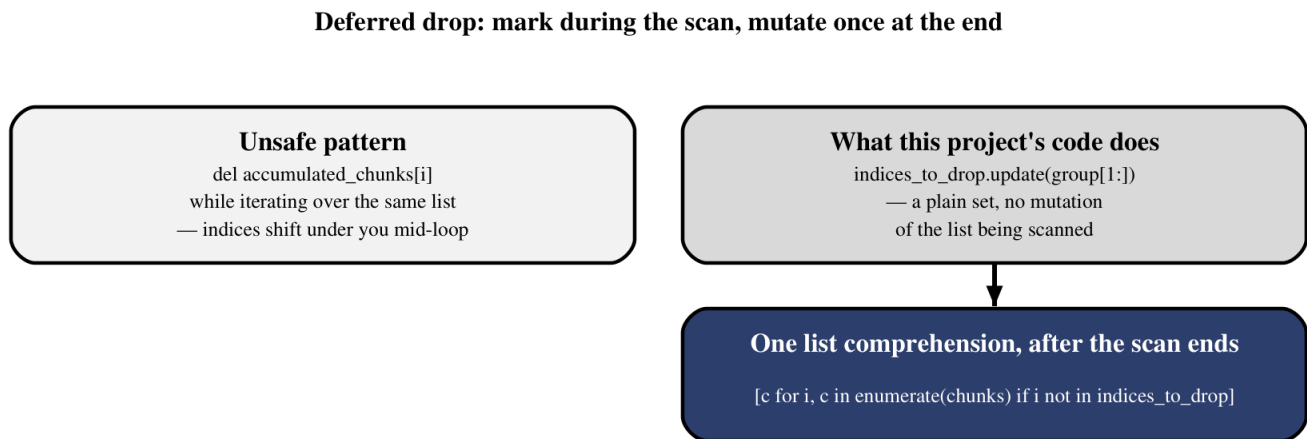


Figure 34.2 — Marking indices for removal during the scan and removing them in one pass afterward avoids the entire class of index-shift bugs a live deletion would risk.

Common pitfall — Trusting an index while its container changes

Any loop that both reads a list by index and removes items from that same list within the loop body is a latent bug, whether or not a specific test case happens to expose it. The deferred-collection pattern — record what to remove, remove it all afterward in one pass — generalizes far beyond chunk merging to any similar scan-and-modify situation.

34.7 Re-embedding merged chunks

`_merge_similar_chunks()`'s return value includes a freshly computed embedding — `"embedding": embedding_manager.generate_embedding(merged_content)` — derived from the **merged** content, not inherited from any of the group's original member chunks. This matters directly for correctness on a run with more than one merge pass: if the agent loops back to RETRIEVE after an INSUFFICIENT verdict (Chapter 16.3) and retrieves more chunks, those new chunks get compared against the **already-merged** pool, and a stale, pre-merge embedding would compare new chunks against content that no longer accurately represents what the merged chunk actually says.

This is the same principle Chapter 32.5's distillation dedup relies on from a different angle — a piece of derived data (a hash there, an embedding here) is only trustworthy if it is recomputed whenever the content it represents changes, never carried forward from a prior version of that content.

The cost of getting this wrong would be silent rather than crash-visible, which is exactly what makes it worth stating explicitly. A stale embedding does not raise an exception — it simply produces a cosine similarity score computed against content that no longer exists in that form, quietly degrading the accuracy of every future merge decision involving that chunk without any error message pointing to the cause.

34.8 Telemetry: logging near-misses to tune the threshold empirically

`MERGE_SIMILARITY_THRESHOLD = 0.90` was set once and, as far as this project's own ledger records, has not been revisited with the same evidence-based rigor Chapter 36C documents for the retrieval thresholds. The gap is worth naming as a concrete opportunity rather than glossing over: logging every **near-miss** comparison — pairs that scored close to but just under 0.90 — would build exactly the kind of empirical distribution Research topic 39 used to justify `DOCUMENTS_MIN_SIMILARITY` and `LEARNED_QA_MIN_SIMILARITY`, but for the merge threshold specifically.

A minimal version costs almost nothing to add: alongside the existing `'sim >= MERGE_SIMILARITY_THRESHOLD'` check, log every comparison whose score falls within some band below the threshold (0.80-0.90, say) with both chunks' content previews. Reviewing that log periodically answers the question Section 34.2's reasoning could only address in the abstract — whether 0.90 is catching the real near-duplicates this project's own corpus and query patterns produce, or leaving genuine redundancy unmerged just below the line.

The methodology this would follow is not an invention — it is Chapter 36C's A/B log-comparison approach, applied one level deeper than that chapter's own retrieval-threshold focus. Where Chapter 36C compares retrieved-chunk sets across two full config values, a merge-threshold near-miss log would compare **pairwise similarity scores** across a single run, which is a smaller and cheaper instrumentation change but follows the identical underlying principle: a threshold chosen once, by reasoning alone, is a hypothesis, and only production data confirms or corrects it.

Chapter 35 moves from the mechanisms themselves to the interface a user or operator actually touches — the CLI commands (`'bad'`, `'stats'`, `'learn'`) that trigger, inspect, and control everything Chapters 29 through 34 built.

Reading this chapter's three pieces — threshold, clustering strategy, deferred mutation — together, the common thread is that none of them is an accident of implementation convenience. A looser or tighter threshold, single-link instead of star-shaped clustering, or a live delete instead of a deferred one would all still **run** without crashing on the common case. What distinguishes this project's actual choices is that each one was made with a specific downstream consequence in mind — faithfulness under LLM merging, safety against silent index corruption — rather than picked for being merely the first approach that happened to pass a quick manual test.